

From the Ground Up

Derivatives $\rightarrow$ Backprop $\rightarrow$ Neural Networks

A Complete Lecture, Built in Layers

10 Sections · From First Principles to Image Classification

Table of Contents

- 1 Derivatives
- 2 The Chain Rule
- 3 Gradient Descent
- 4 Gradients (Many Inputs)
- 5 The Autograd Engine
- 6 Backprop
- 7 Neural Networks
- 8 The Loss

What is a Derivative?

The Question: Given $f(x)$, how should we change x to make $f(x)$ smaller?

The Definition:

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

"If I nudge x by a tiny amount, how much does f change, per unit of nudge?"

Examples:

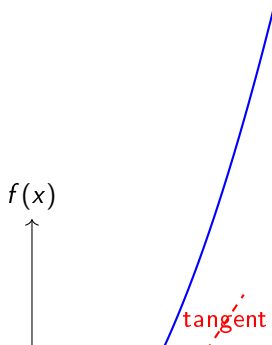
$f(x)$	$f'(x)$	Meaning
x^2	$2x$	at $x = 3$, slope is 6
$3x + 1$	3	constant slope
$\sin(x)$	$\cos(x)$	oscillates
e^x	e^x	slope equals value

The Linear Approximation

$$f(x + h) \approx f(x) + f'(x) \cdot h$$

“New value $\approx$ old value + (slope) $\times$ (how far I moved).”

This is the **only** first-order approximation matching f at x with correct slope. It is what makes derivatives useful.



The Learning Rule

$$x_{\text{new}} = x - \eta \cdot f'(x)$$

- If $f'(x) > 0$: increasing x increases $f \implies$ **decrease** x
- If $f'(x) < 0$: increasing x decreases $f \implies$ **increase** x
- η (learning rate) is a small positive number

The sign tells direction; the magnitude tells steepness.

$$x_{\text{new}} = x - \eta \cdot f'(x) \quad \longleftarrow \quad \text{This is gradient descent in 1D}$$

Compositions

Real functions are compositions:

$$u = 3x + 1, \quad v = u^2, \quad f = v + 5$$

How does x affect f ?

The Rule:

$$\frac{df}{dx} = \frac{df}{dv} \cdot \frac{dv}{du} \cdot \frac{du}{dx}$$

Multiply the local derivatives along the path.

Intuition: Think of each derivative as a ratio of small changes:

$$\frac{\Delta f}{\Delta x} = \frac{\Delta f}{\Delta v} \cdot \frac{\Delta v}{\Delta u} \cdot \frac{\Delta u}{\Delta x}$$

The intermediate differentials cancel like fractions

Concrete Example

With $x = 2$:

$$u = 7, \quad \frac{du}{dx} = 3 \quad v = 49, \quad \frac{dv}{du} = 14 \quad f = 54, \quad \frac{df}{dv} = 1$$

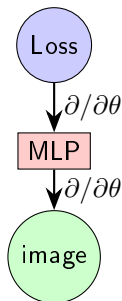
$$\frac{df}{dx} = 1 \cdot 14 \cdot 3 = 42$$

Check directly: $f(x) = (3x + 1)^2 + 5$, so $f'(x) = 6(3x + 1) = 6 \cdot 7 = 42$. ✓

Branching: If f depends on x through multiple paths, **sum** the contributions:

$$f = g(x) + h(x) \implies \frac{df}{dx} = \frac{dg}{dx} + \frac{dh}{dx}$$

Why This Is the Whole Game



Every neural network is a deeply nested composition.

The chain rule lets us compute $\partial \text{Loss} / \partial \theta$ for every parameter θ by multiplying local derivatives along the path from θ to the loss.

The Optimization Problem

We want to minimize a loss function:

$$\theta^* = \arg \min_{\theta} L(\theta)$$

We can't solve analytically (nonlinear, millions of parameters). But we can:

- Compute $\frac{dL}{d\theta}$ at current θ
- Take a small step in the **opposite** direction
- Repeat

The Update Rule:

$$\theta \leftarrow \theta - \eta \cdot \frac{dL}{d\theta}$$

- $dL/d\theta > 0$: decrease θ
- $dL/d\theta < 0$: increase θ

The Learning Rate η

$$\theta \leftarrow \theta - \eta \cdot \frac{dL}{d\theta}$$

η too small	Training crawls
η too large	Overshoot, oscillate, diverge
η just right	Smooth descent to local minimum

Typical values: 0.001 to 0.1.

It is the single most important hyperparameter.

Why it works: For small η , the linear approximation holds. Each step provably decreases L (up to second-order terms). Gradient descent is **local search guided by a global mathematical tool**.

From Scalar to Vector

When L depends on many parameters $\theta = (\theta_1, \dots, \theta_n)$:

$$\nabla L = \left[\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_n} \right]$$

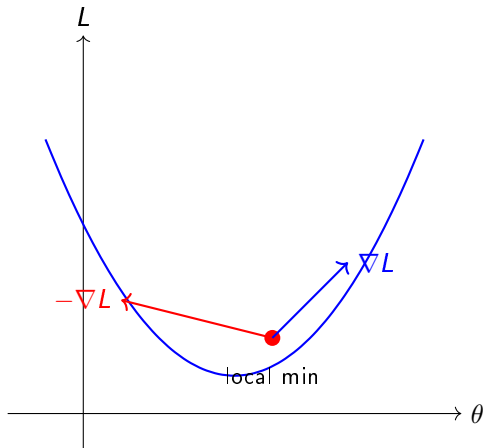
Each component answers: *"If I nudge only θ_i , how much does L change?"*

Elementwise Update:

$$\theta_i \leftarrow \theta_i - \eta \cdot \frac{\partial L}{\partial \theta_i} \quad \text{for every } i$$

The Gradient Points Uphill

Key fact from multivariable calculus: ∇L points in the direction of **steepest ascent**. So $-\nabla L$ points downhill.



The Big Realization

For a neural network, θ can have **millions** of components.

Naive approach: Perturb each θ_i slightly, recompute loss $\implies N$ forward passes for N parameters. *Prohibitively expensive.*

Backprop: Computes **all** gradients in **one backward pass**.

Cost $\approx 2 \times$ forward (independent of parameter count)

This is the trick that makes deep learning feasible.

The Value Class

Every number in our computation is a Value. Each remembers:

- Its **data** — the number
- Its **gradient** $\partial L / \partial \text{self}$ (starts at 0)
- How it was **created** — the operation and inputs
- **`_backward`** — how to propagate gradient backward

```
1 class Value:
2     def __init__(self, data, _children=(), _op=''):
3         self.data = data
4         self.grad = 0.0
5         self._backward = lambda: None
6         self._prev = set(_children)
7         self._op = _op
8
```

Listing 1: The Value class skeleton

Local Derivatives as Code

Addition: $c = a + b \implies \partial c / \partial a = 1, \partial c / \partial b = 1$

```
1 def __add__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data + other.data, (self, other), '+')
4     def _backward():
5         self.grad += 1.0 * out.grad
6         other.grad += 1.0 * out.grad
7     out._backward = _backward
8     return out
9
```

Multiplication: $c = a \cdot b \implies \partial c / \partial a = b, \partial c / \partial b = a$

```
1 def __mul__(self, other):
2     other = other if isinstance(other, Value) else Value(other)
3     out = Value(self.data * other.data, (self, other), '*')
4     def _backward():
5         self.grad += other.data * out.grad
6         other.grad += self.data * out.grad
7     out._backward = _backward
8     return out
```

Why +=?

If a value is used multiple times, gradients from all uses must **sum**.

$$f = a \cdot b + a \cdot c \implies \frac{\partial f}{\partial a} = b + c$$



+= accumulates both contributions to a 's gradient.

Forward Pass Builds a Graph

```
1 loss = ((y_pred - y) ** 2).tanh()  
2
```

Listing 2: Writing expressions builds a Value DAG

Python constructs a tree of Value objects. Each has `_prev` pointing to its inputs.
The graph is the computation.

Backward Pass:

```
1 def backward(self):  
2     topo, visited = [], set()  
3     def build(v):  
4         if v not in visited:  
5             visited.add(v)  
6             for c in v._prev: build(c)  
7             topo.append(v)  
8     build(self)  
9     self.grad = 1.0  
10    for v in reversed(topo):  
11        v._backward()  
12
```

The Setup

loss (root) $\longleftarrow$ operations $\longleftarrow$ parameters θ (leaves)

Goal: compute $\partial \text{loss} / \partial \theta$ for every leaf.

Topological Order: Process each node *after* every node that depends on it. Visiting in **reverse** topological order means the root comes first, then its consumers, then down to the leaves.

Seeding: `loss.grad = 1.0`. Because $\partial \text{loss} / \partial \text{loss} = 1$. Every other gradient is relative to this.

The Reverse Walk

For each node, calling `_backward()`:

- Reads `node.grad` (incoming gradient from all consumers — already accumulated)
- Multiplies by each local derivative
- Adds results to each input's `.grad`

This is the chain rule, evaluated in reverse, with accumulation at every node.



reverse order

Why This Is Efficient

Naive: N forward passes for N parameters Backprop: 1 forward + 1 backward ≈ 2

The chain rule is reused: intermediate results (activations, local derivatives) are computed **once** and shared across all paths that flow through them.

Backprop = Reverse-Mode Autodiff

- **Reverse mode:** we walk the graph backward from the output
- **Auto-diff:** we don't write gradients by hand — the graph is differentiated automatically

The Full Picture

save inputs & value

`value = op(inputs)`

for each op in reverse order

for each input: `input.grad += local_deriv × output.grad`

A Neuron

A neuron is a small differentiable function:

$$h = \tanh(w \cdot x + b)$$

```
1 class Neuron:
2     def __init__(self, nin):
3         self.w = [Value(random.uniform(-1, 1) / (nin ** 0.5))
4                     for _ in range(nin)]
5         self.b = Value(0.0)
6
7     def __call__(self, x):
8         act = self.b
9         for xi, wi in zip(x, self.w):
10             act = act + xi * wi
11         return act.tanh()
12
```

$1/\sqrt{n}$ **init** keeps the dot product's magnitude stable as n grows

Why Nonlinearity?

Without $\tanh$ (or ReLU), a stack of linear layers collapses into a single linear function:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2) = Wx + b$$

No matter how deep, it's still linear. Nonlinearity is what lets deep networks represent nonlinear functions.

Linear



Linear

$= Wx + b$ (still linear!)

A Layer

A layer is n_{out} neurons, each seeing the same input:

$$\text{Layer}(x) = [\text{Neuron}_1(x), \dots, \text{Neuron}_{n_{\text{out}}}(x)]$$

```
1 class Layer:
2     def __init__(self, nin, nout):
3         self.neurons = [Neuron(nin) for _ in range(nout)]
4
5     def __call__(self, x):
6         outs = [n(x) for n in self.neurons]
7         return outs[0] if len(outs) == 1 else outs
8
```


The MLP

```
1 class MLP:
2     def __init__(self, nin, nouts):
3         sizes = [nin] + nouts
4         self.layers = [Layer(sizes[i], sizes[i+1])
5                           for i in range(len(nouts))]
6
7     def __call__(self, x):
8         for layer in self.layers:
9             x = layer(x)
10        return x
11
```

For classification, we typically ***do not*** apply `tanh` on the last layer — we want raw logits for softmax cross-entropy.

Parameter Count

For MLP(64, [32, 32, 5]):

Layer	Formula	Count	Cumulative
Layer 1	$32 \times (64 + 1)$	2080	2080
Layer 2	$32 \times (32 + 1)$	1056	3136
Layer 3	$5 \times (32 + 1)$	165	3301
Total		3301	

This number is **fixed by architecture**, not by dataset size. Whether we feed 100 images or 100 million, the network has 3301 numbers to learn.

Every Forward Pass Is a Composition

Given input x :

- Compute $w_1 \cdot x + b_1$, apply $\tanh$
- Compute $w_2 \cdot h_1 + b_2$, apply $\tanh$
- Compute $w_3 \cdot h_2 + b_3$
- Return logits

Every step is Value operations. The entire forward pass produces a graph of thousands of Value nodes. Backprop walks it in reverse **once**, and every parameter's gradient is filled in.

Cross-Entropy for Classification

For logits $\mathbf{z} = [z_1, \dots, z_C]$ and target class t :

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad \text{loss} = -\log(\text{softmax}(\mathbf{z})_t)$$

The model predicts a probability distribution. Loss penalizes low probability on the correct class.

Why Log + Softmax?

The gradient w.r.t. the logits is beautiful:

$$\frac{\partial \text{loss}}{\partial z_i} = \text{softmax}(z)_i - \mathbb{1}[i = t]$$

Predicted probability — one-hot target. Well-scaled, never vanishing, never exploding.

We get this automatically — backprop derives it. We never write it by hand. The point of autograd is correct gradients regardless of loss complexity.

Why not MSE? tanh saturates. When confidently wrong, gradients vanish. MSE gradients on probabilities are small near 0 or 1.

Numerical Stability

e^z overflows for large z . Standard trick: subtract the max before exponentiating. Softmax is invariant to this shift.

```
1 def softmax_cross_entropy(logits, target_idx):
2     max_val = max(z.data for z in logits)
3     shifted = [z - max_val for z in logits] # stable
4     exps = [s.exp() for s in shifted]
5     total = exps[0]
6     for e in exps[1:]:
7         total = total + e
8     probs = [e / total for e in exps]
9     return -probs[target_idx].log()
10
```

Backprop handles this composition — exp, division, log — automatically.

The Training Loop

```
1 # --- Forward ---
2 batch_loss = Value(0.0)
3 for img, label in batch:
4     x = [Value(p) for p in img]
5     logits = model(x)
6     batch_loss = batch_loss + softmax_cross_entropy(logits, label)
7 batch_loss = batch_loss * (1.0 / BATCH_SIZE)
8
9 # --- Backward ---
10 for p in params: p.grad = 0.0
11 batch_loss.backward()
12
13 # --- Update ---
14 for p in params:
15     p.data -= LEARNING_RATE * p.grad
16
```

Listing 4: Complete training step per batch

- **Zero gradients:** += accumulates, so yesterday's values must be cleared
- **Backward:** topological sort $\rightarrow$ seed $\rightarrow$ reverse walk
- **Update:** $\theta_i \leftarrow \theta_i - \eta \cdot \partial L / \partial \theta_i$

Build graph; [draw, fill=orange!30, minimum width=2.5cm, right=of fwd] (bwd)

Backward

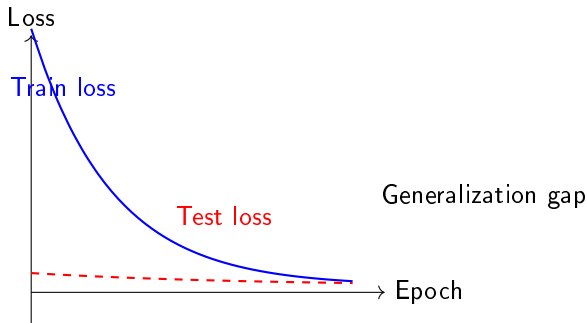
Compute grads; [draw, fill=blue!20, minimum width=2.5cm, right=of bwd] (upd)

Update

Step downhill;

```
[-Stealth[length=3mm], thick] (fwd) -- node[above]build (bwd);
[-Stealth[length=3mm], thick] (bwd) -- node[above]grads (upd);
[-Stealth[length=3mm], thick, dashed] (upd) to[out=270,in=90]
node[below]repeat (fwd);
```


What Changes as the Model Learns



Early: loss decreases rapidly. **Middle:** useful features emerge. **Late:** slow decrease. If train and test loss diverge $\Rightarrow$ **overfitting**. The **generalization gap** is the central tension in ML.

The Dataset

Synthetic 8×8 images of digits 0–4, drawn as crude strokes + 10% noise.
Flattened to 64 pixels.

Why no convolutions? Our goal is to understand backprop, not build a state-of-the-art model. Convs add new ops with their own backward rules, obscuring core ideas.

Key insight: The backprop algorithm is **identical** with or without convolutions. Convs are just one more differentiable op. We flatten. The model loses spatial structure, but gradients remain transparent.

Results

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
0	1.5861	0.281	1.5618	0.300
10	0.2210	0.881	0.3412	0.775
20	0.0781	0.963	0.2218	0.850
39	0.0281	0.991	0.1540	0.925

Train accuracy climbs to 99%. Test accuracy plateaus at 92.5%. **The model learns.**

Reshaping first-layer weights to 8×8 reveals stroke detectors: horizontal bars, verticals, diagonals. The second layer combines these into digit-part detectors. **This hierarchy emerges automatically from gradient descent.**

We never told the model “look for strokes.” We told it “minimize the loss,” and backprop did the rest.

What We Skipped (Refinements)

Feature	Trade-off
Convolutions	Restore spatial structure, more op complexity
Vectorization (NumPy)	100× faster, hides elementwise ops
Better optimizers (Adam)	Same backprop, better gradient use
Regularization (dropout)	Improve generalization, not optimization

All are refinements on the same core loop:

forward → backward → step

The math is unchanged.

In One Paragraph

Every parameter in a neural network has a derivative: how the loss changes when you nudge that parameter.

Gradient descent uses these derivatives to step each parameter downhill.

The chain rule tells us how to compute the derivative of a composition — multiply local derivatives along the path.

Backprop is the chain rule applied systematically: build a graph on the forward pass, walk it backward to compute every parameter's gradient in one pass.

A neural network is a composition of simple differentiable functions — linear combinations and nonlinearities.

Training is a loop: forward to compute loss, backward to compute gradients, update to nudge parameters.

Everything else — convs, attention, transformers — is a new differentiable op with a new local derivative. The engine never changes.

Exercises

1. **Trace by hand.** 2-input neuron, compute forward and backward. Verify with code.
2. **Break +=.** Replace with = in `__add__`'s backward. Watch gradients come out wrong.
3. **Add ReLU.** Derive its local derivative. Compare convergence to tanh.
4. **Remove gradient zeroing.** See training diverge. Understand why.
5. **Try MSE + tanh on classification.** Observe slow early training.
6. **Print the graph.** Walk `_prev` from loss node. See the DAG.
7. **Swap in NumPy.** Same math, $100\times$ faster. New complexity: broadcasting.
8. **Add a conv.** 3×3 filter sliding over image. Derive backward. See accuracy jump.

Thank You!

Derivatives $\rightarrow$ Chain Rule $\rightarrow$ Backprop $\rightarrow$ Neural Networks

That's backprop from the ground up.